

Plataforma unificada

Envolver StrategyQuant X y AlgoProject en un solo producto, con una interfaz propia, una cola de trabajo común y progreso en tiempo real.

Generado el 2026-09-20 · sobre el árbol en `master`

Entorno medido · Linux, 96 núcleos · StrategyQuant X build 2953 · 16 proyectos · 33.629 líneas de Python · 2,3 GB de datos

Naturaleza · viabilidad y plan. No se ha escrito código del producto, ni se ha modificado SQX, ningún proyecto ni ningún databank

Los hallazgos permanentes viven en `knowhow/03-driving-sqx.md`

Plataforma unificada — envolver SQX y AlgoProject en un solo producto

Qué es este documento. Un estudio de viabilidad. Responde a una pregunta concreta: *¿se puede construir un software propio, con una interfaz de calidad profesional, que contenga tanto a StrategyQuant X como a las funcionalidades de AlgoProject, de forma que el trabajo diario deje de ser "abrir SQX, correr todo, cambiar de ventana, correr todo otra vez"?*

La respuesta es **sí**, y este documento dice por qué, con qué piezas, qué falta, qué cuesta y qué no se ha probado todavía.

Qué NO es. No es un plan de implementación aprobado, ni una especificación. No se ha escrito ni una línea de código del producto. Nada de lo que aparece aquí ha modificado SQX, ningún proyecto, ningún databank ni ningún fichero de configuración. Todo lo empírico se obtuvo leyendo: ficheros en disco, logs, el JavaScript de la propia GUI de SQX, y una única petición HTTP de solo lectura.

Por qué existe. El equipo está todavía terminando los módulos de análisis básicos y su perfilado. Esta conversación no se va a ejecutar ahora — pero lo que se descubrió en ella, especialmente el hallazgo de la sección 5, no puede perderse en un transcript.

Estado del entorno medido. Máquina Linux de 96 núcleos. StrategyQuant X build 2953, instalación maestra en `~/Desktop/SQX` con la GUI levantada durante toda la investigación, worker headless en `~/Desktop/SQX_w1` (apagado). 16 proyectos en el maestro. AlgoProject en `master`, 33.629 líneas de Python. Datos: 2,3 GB en `~/Desktop/AlgoData`.

Generado el 2026-09-20.

1. El problema, en los términos del equipo

AlgoProject funciona. Hace matemáticas que SQX no hace, y las hace bien. Pero se usa así:

"Hay que ejecutar comando a comando. Los distintos módulos de análisis — Monte Carlo, retest, etc. — están separados: cada uno tiene su propio panel, con su propia interfaz. Y lo más pesado de todo: hay que abrir SQX, correr todo, y después cambiar a AlgoProject y correr todo aquí de nuevo."

Tres problemas distintos escondidos en una sola queja:

#	problema	naturaleza
P1	La interfaz es una terminal	de presentación
P2	Cada estudio es una aplicación aparte, sin estado compartido	de arquitectura
P3	SQX y el análisis son dos mundos que el humano conecta a mano	de integración

P3 es el caro, y es el que de verdad duele. P1 se arregla con diseño. P2 con un proceso común. P3 exige que SQX deje de ser una aplicación que se abre y pase a ser un módulo que se invoca.

Aclaración recogida del equipo, y que acota mucho el alcance:

"No necesitamos envolver visualmente SQX. SQX es un módulo más de nuestro software (un módulo muy grande y complejo), pero no queremos sustituirlo visualmente. Como mucho, querríamos que al pulsar un botón en nuestra aplicación se ejecute el correspondiente conjunto de órdenes en SQX —por ejemplo, un Walk Forward— y, a ser posible, ver una barra de progreso en nuestra aplicación."

Eso convierte un proyecto grande y arriesgado en uno acotado y medible. El resto de este documento responde a esa versión de la pregunta.

2. Inventario: qué existe ya

Esto importa porque cambia la conversación de "construir un producto" a "poner carcasa a un motor que ya funciona". Medido el 2026-09-20 sobre el árbol en `master`:

área	ficheros <code>.py</code>	líneas	qué es
<code>strategies/</code>	121	14.418	análisis de una estrategia: Monte Carlo, retest, cross-market
<code>sqx/</code>	17	1.897	hablar con SQX: exportar, inspeccionar, reparar
<code>core/</code>	16	1.602	librería compartida: formatos, rutas, estadística
<code>perf/</code>	19	1.522	instrumentación y catálogo de rendimiento
<code>tasks/</code>	10	1.084	análisis de poblaciones enteras
<code>tools/</code>	4	545	checks, mapa de dependencias, manual
<code>tests/</code>	5	370	tests, incluido un golden test con fixture
total	252	33.629	(incluyendo el plugin <code>sqx-lab</code>)

Más allá del código:

- **33 puntos de entrada propios** (`python3 -m ...`), cada uno con `--help`.
- **3 paneles web ya escritos** — `monteCarlo`, `retest`, `crossmarket` — con servidor Flask, cola de trabajos, caché de resultados, tooltips y cajón de configuración. Unas 4.700 líneas entre los tres, de las cuales ~1.100 son HTML de página.
- **14 skills de agente** (`analysis-montecarlo`, `analysis-retest`, `analysis-crossmarket`, `analysis-generation`, `export`, `translate`, `perf`, `audit` ... más las cuatro de `sqx-lab`).
- **9 ficheros de knowhow** y **17 páginas de manual** en español, con PDF generado.
- **2,3 GB de datos** en `~/Desktop/AlgoData`, con tres zonas de semántica distinta (`metrics/` se sobrescribe, `raw/` es inmutable, `reports/` se acumula) y un `INDEX.md` que cataloga lo que existe.

Dos propiedades del código son las que hacen viable el producto, y conviene nombrarlas porque no son accidentales:

1. **`core/paths.py` es el único módulo que sabe dónde vive nada.** Ninguna ruta absoluta fuera de él; `config/machine.yaml` es el único fichero que cambia entre máquinas. Un demonio puede montarse encima sin tocar la lógica.

2. **Las skills ya son, de facto, la API de alto nivel.** `analysis-montecarlo` no es documentación: es la definición ejecutable de qué significa "hazme un Monte Carlo de este databank". Esa capa normalmente hay que inventarla. Aquí está escrita.

Conclusión del inventario: el backend del producto existe. Falta la carcasa, un proceso de estado común y un lenguaje visual único.

3. La fricción, paso a paso

El día de trabajo actual, para un solo estudio, tal y como lo describe el manual:

1. Abrir SQX (GUI)
2. Lanzar el proyecto / la tarea — minutos u horas, mirando la barra de SQX
3. Esperar a que termine (y acordarse de volver a mirar)
4. Cambiar de ventana → terminal
5. `python3 -m sqx.export.export_metrics ...`
6. `python3 -m sqx.export.export_trades ...`
7. `python3 -m strategies.monteCarlo.explorer.serve --project X --databank Y`
8. Se abre el navegador con el panel de Monte Carlo
9. ¿Y el retest? Otra terminal, otro comando, OTRO panel, otra pestaña
10. ¿Y cross-market? Lo mismo otra vez
11. Cruzar a ojo los resultados de tres paneles que no se conocen entre sí

Once pasos, tres ventanas, dos aplicaciones y un humano haciendo de bus de datos entre ellas. Y el paso 3 — esperar sin saber cuánto— es el que rompe el ritmo de trabajo.

Nótese que **nada de esto es un defecto del código**. Cada pieza hace bien su trabajo. El defecto está en el espacio entre las piezas, que hoy lo ocupa una persona.

4. Hallazgo: SQX es una aplicación web dentro de un Electron

Este es el hallazgo que cambia el proyecto de "difícil" a "acotado", y se encontró buscando de dónde sacar la barra de progreso.

▮ **StrategyQuant X no tiene interfaz nativa.** Su GUI es una aplicación web servida por un Jetty embebido y renderizada en un Electron. En disco, cada módulo es una carpeta de HTML y JavaScript:

```
internal/electron/                                la carcasa
internal/web/BUILDER    RETESTER    TASKMANAGER    RESULTS    RESULTS2
                        OPTIMIZER    PORTFOLIOMASTER    PORTFOLIOCOMPOSER    AlgoWizard
                        SQWIZARD    SQMANAGER    QDM    MTANALYZER    NEURALNETWORK    ...
```

▮ Confirmado en el log de arranque del propio maestro, que registra `com.strategyquant.webguilib.Electron`, `c.strategyquant.webguilib.BrowserGUI`, `c.s.webguilib.servlet.MainServlet` y `org.eclipse.jetty.server.Server` (jetty-all-uber 11.0.20).

Por qué importa tanto. La disyuntiva no era, como parecía, "o *pilotamos la CLI*, o *automatizamos una GUI nativa*". Automatizar una GUI nativa (Java/Swing, clics sintéticos, reconocimiento de ventanas) es lo caro, lo frágil y lo que se rompe en cada actualización. Pero **la interfaz de SQX ya es HTTP + WebSocket sobre localhost**. Todo lo que un navegador puede hacerle, puede hacérselo código.

Dicho de otro modo: SQX ya es, por dentro, exactamente el tipo de aplicación que el equipo quiere construir. No hay que domesticar un programa ajeno; hay que hablar con un servidor.

5. La barra de progreso: dos rutas, y las dos funcionan

Ruta A — el mismo WebSocket que usa SQX para pintar sus propias barras

□ **SQX no hace polling de progreso: emite eventos.** En `internal/web/common/Batch1/Libs.js`:

```
C("/main/getWebSocketPort", {}, "GET", function (e) {
  x("ws://" + window.location.hostname + ":" + e.port + "/websocket/updates",
    "WebSocketMessageMain")
})
```

La secuencia es: `GET /main/getWebSocketPort` → `{"port": N}` → conectar a `ws://localhost:N/websocket/updates`. El vocabulario del cliente alrededor de ese canal incluye `progressChannel`, `progressUpdateEvent`, `progressPercent`, `progressAction` y `progressText` — exactamente los campos que dibujan las barras de progreso de SQX.

⚠ **En esta instalación el endpoint está cerrado.** Con la GUI del maestro levantada:

```
$ curl -s http://localhost:8080/main/getWebSocketPort
{"disabled":true,"error":"Remote access disabled"}
```

Hay una opción de acceso remoto en la configuración de SQX que lo gobierna. □ **No se ha probado** qué expone al abrirla: si el `/main/*` completo o solo la búsqueda de puerto. Es la única incógnita real de todo este documento, y se resuelve en una tarde. Es también la puerta que separa "control por comandos" de "barra de progreso en tiempo real idéntica a la de SQX".

Ruta B — el log, que funciona hoy y sin tocar ninguna configuración

□ Todo evento de tarea pasa por un único logger, `c.s.t.project.ProgressEngine`, con un vocabulario estable y parseable:

```

<TAREA> : =====
<TAREA> : Starting strategies retesting...
<TAREA> : Loading backtest data for Main test - <SÍMBOLO> / <TF>
<TAREA> : All backtest data prepared
<TAREA> : Sequential optimization: <ESTRATEGIA> - Optimizing parameter <NOMBRE>...
<TAREA> : Task finished in N.N s.
Project finished
Databank '<PROY>/<BANCO>' loaded - N strategies in Nms
Databank '<PROY>/<BANCO>' saved - files before sync N / after sync N / saved N / removed N in N s.

```

▮ Y el porcentaje fino no viaja en el mensaje, sino **en el nombre del hilo**:

```
[Blocking computeThread common #45 - WF: 6 runs : 20 % 00S WFO 4]
```

Haciendo `tail` de `user/log/StrategyQuant/log_<fecha>.log` se obtienen las dos cosas: frontera de tarea y porcentaje. Sin habilitar nada, sin riesgo para la instancia en marcha, y funciona igual en el maestro con GUI que en un worker headless.

⚠ Ese log es grande y se escribe rápido: **55 MB a las 20:44 del 2026-09-20**. Se tailea; nunca se lee entero.

Ruta C — el recuento, como respaldo grosero

Para tareas que llenan un databank, `-databank action=count` en un temporizador da una medida de avance honesta ("237 de ~1.200 estrategias") sin parsear nada.

Veredicto: la barra de progreso no es el riesgo del proyecto. Hay tres rutas, dos de ellas disponibles hoy.

6. La superficie de control de SQX

Consolidado de `internal/web/SQUANT/help.txt` (la referencia completa de verbos de `sqcli`, legible sin arrancar SQX) y de lo verificado en `knowhow/03-driving-sqx.md`.

Puertos

	puerto	¿funciona con la GUI del maestro levantada?
API de comandos del maestro	5050	▮ <code>Error: CLI not ready.</code>
MCP / web del maestro	8080	▮ pero solo lectura + run/stop; <code>/call</code> devuelve 404
API de comandos del worker	5060	▮ siempre — headless, sin GUI con la que competir

Lo que se puede ordenar

quiero	ruta	¿con GUI del maestro levantada?
Lanzar una cadena de tareas	<code>-project action=start</code>	solo worker
Lanzar UNA tarea concreta (p. ej. Walk Forward)	<code>-project action=startOnlyTask name=X task=N</code>	solo worker
Lanzar desde una tarea en adelante	<code>-project action=startFromTask</code>	solo worker
Parar	<code>-project action=stop · MCP stop_project</code>	worker · maestro ☐
Pausar / reanudar	<code>-project action=pause / resume</code>	solo worker
Estado de la tarea	<code>-project action=status name=X</code>	solo worker
Crear / modificar un proyecto	<code>-project action=loadconfig</code>	solo worker
Listar, contar, exportar, limpiar databanks	<code>-databank action=...</code>	solo worker
Exportar operaciones a CSV	<code>-tools action=orderstocsv</code>	solo worker
Exportar barras	<code>-data action=export</code>	solo worker
Progreso en vivo	<code>/websocket/updates</code>	requiere abrir el acceso remoto
Progreso en vivo, sin tocar nada	<code>tail de ProgressEngine</code>	☐ siempre

El "botón Walk Forward" que pide el equipo es, literalmente, `startOnlyTask` con el número de tarea correcto. La dificultad no está ahí. Está en las cuatro restricciones siguientes.

7. Las cuatro restricciones que dictan la arquitectura

No son opinables. Están verificadas y documentadas en `knowhow/`.

R1 — Con la GUI del maestro abierta, su CLI está muerta. Devuelve `CLI not ready`. Solo responde el worker headless en el 5060. *Consecuencia de diseño: la aplicación no pilota la ventana del propietario; gestiona sus propias instancias headless.*

R2 — El MCP del maestro es de solo lectura más run/stop. No existe verbo de escritura de configuración. Meter un proyecto en el maestro vivo solo se puede por su propio menú. *Consecuencia: hay operaciones que seguirán siendo manuales, y el producto tiene que admitirlo sin fingir.*

R3 — Cada sync de SQX borra del disco los `.sqx` que no tiene en memoria. Del log del maestro:

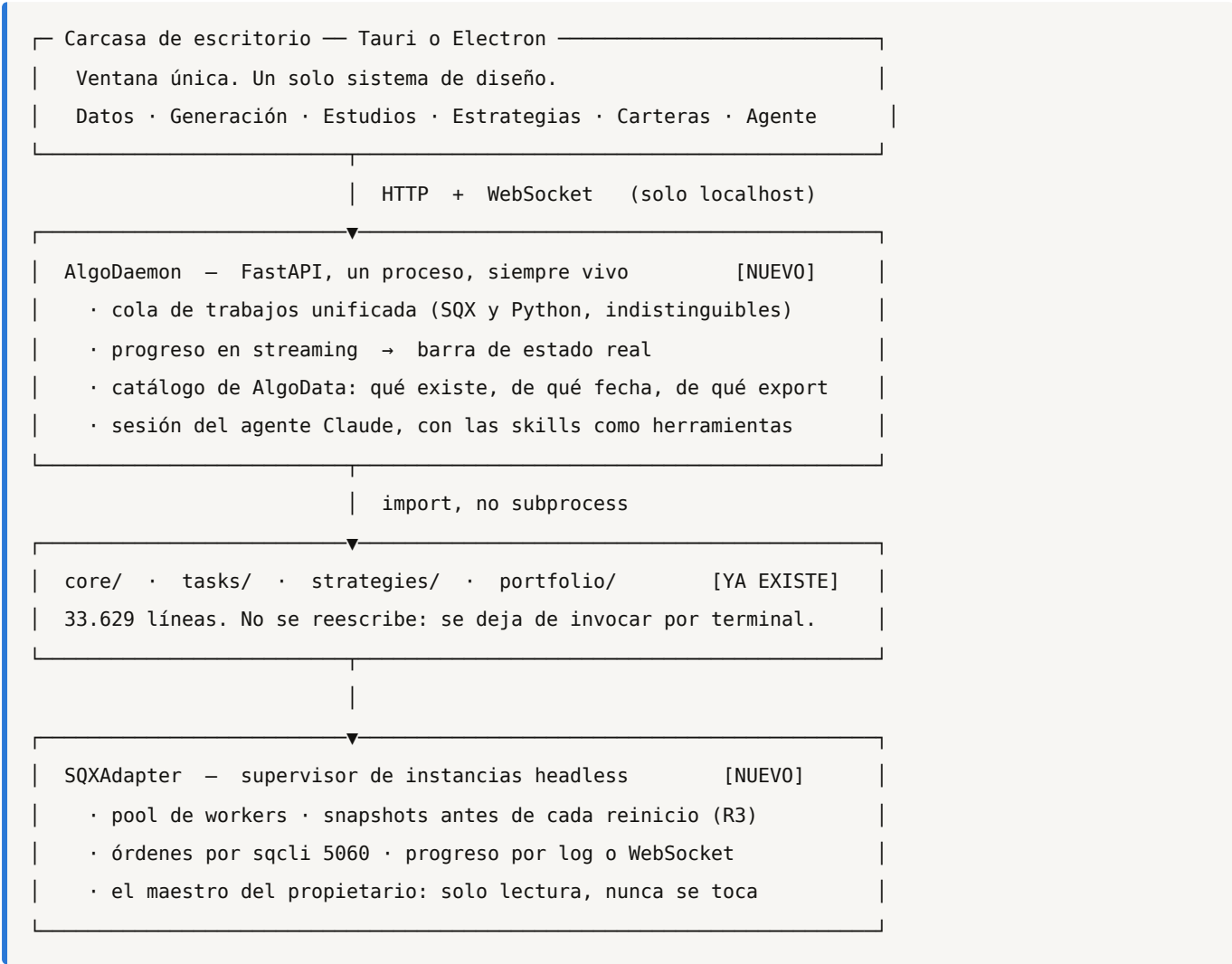
```
08:11:31 'Project - USDJPY/WFM' before sync 248 / after sync 36 / saved 36 / removed 248
08:24:58 'Project - XAUUSD/WFM' before sync 1208 / after sync 1142 / saved 1142 / removed 66
```

La primera línea es un auto-sync horario, no un apagado. *Consecuencia: un orquestador ingenuo destruye trabajo. Hace falta un supervisor de ciclo de vida con snapshots, no un lanzador de procesos. Esta es la parte técnicamente delicada del proyecto entero.*

R4 — bin/sqx-worker.sh es bash. Necesita rsync , ss , curl y setsid . Hoy la mitad exportadora solo corre en Linux; la mitad analítica (que lee CSVs) corre en cualquier sitio. *Consecuencia: si el producto tiene que salir de esta máquina, hay que reescribirlo en Python. Es un día de trabajo, pero decidirlo tarde duele.*

8. Arquitectura propuesta

Cuatro capas. Solo la primera y la tercera son trabajo nuevo de verdad.



El cambio conceptual más importante: hoy cada panel es un servidor. En el producto, los paneles dejan de ser servidores y pasan a ser **vistas** sobre un demonio común. Eso es exactamente lo que hace que Monte Carlo, Retest y Cross-market dejen de ser tres aplicaciones y pasen a ser tres pestañas que comparten selección de estrategia, filtros, escala de color y portapapeles.

9. El modelo de trabajo unificado

El demonio expone **una sola noción de "trabajo"**. A la interfaz le da igual si un trabajo es un Walk Forward dentro de SQX o un Monte Carlo en Python: los dos son una fila en la misma cola, con el mismo porcentaje, el mismo botón de cancelar y el mismo aviso al terminar.

```

job = daemon.submit(SQXTask(project="XAUUSD", task=14)) # el botón "Walk Forward"
job.progress → flujo 0..100 # ProgressEngine, o el WebSocket
job.on_done → export + análisis # ← aquí está el premio

```

on_done es la pieza que elimina P3. Hoy terminas en SQX y *empiezas a mano* en AlgoProject. En el producto, el final de la tarea SQX es el disparador del análisis. Los once pasos de la sección 3 se convierten en:

1. Pulsar "Walk Forward" en XAUUSD
2. (opcional) Mirar la barra
3. Volver cuando hay conclusiones

Las cadenas se declaran, no se teclean:

```

receta: walk_forward_completo
- sqx: startOnlyTask project=XAUUSD task=14
- export: metrics + trades del databank WFM
- python: strategies.monteCarlo sobre lo exportado
- python: strategies.retest sobre lo exportado
- vista: panel unificado, pestaña Robustez

```

Eso convierte el producto en algo que un gestor puede usar sin saber qué es un databank — que es, en el fondo, lo que significa "a lo hedge fund".

10. La interfaz: qué pantallas, y por qué

No "una pestaña en el navegador": una ventana de escritorio con navegación lateral persistente, estado global visible y densidad de información alta. Seis zonas:

zona	contiene	sustituye a
Datos	catálogo de <code>AlgoData</code> : qué exports existen, de qué fecha, cuánto ocupan, qué está rancio	leer <code>INDEX.md</code> a mano
Generación	proyectos SQX, sus tareas, botón por tarea, cola y progreso	abrir SQX y navegar sus menús
Estudios	Monte Carlo, Retest, Cross-market como pestañas de una misma vista , con la selección compartida	tres <code>serve.py</code> y tres pestañas de navegador
Estrategias	ficha por estrategia: métricas, operaciones, veredictos de cada estudio, traducción a Python	cruzar tres paneles a ojo
Carteras	composición, correlaciones, riesgo agregado	<code>portfolio/</code> (hoy vacío)
Agente	panel lateral persistente, con las skills como herramientas, viendo el mismo estado que tú	otra terminal, otro contexto

Cuatro decisiones de diseño que conviene fijar el día uno, porque son caras de cambiar después:

1. **Una sola barra de estado global.** Todo lo que corre —SQX o Python— se ve en el mismo sitio. Si el usuario tiene que buscar dónde está el progreso, el producto ha fallado.
2. **La selección es global.** Elegir la estrategia `17.9.39` en un sitio la elige en todos. Es lo que convierte tres paneles en uno.
3. **Una sola escala de color, definida una vez.** Hoy cada panel tiene la suya. En un producto, "rojo" tiene que significar lo mismo en las seis zonas.
4. **Todo número visible se explica al pasar por encima.** Los paneles actuales ya tienen `tooltips.py`; esa disciplina se hereda, no se reinventa.

11. Sobre "envolver" SQX: las tres opciones, y el veredicto

Aunque el equipo ya ha descartado la sustitución visual, conviene dejar las opciones por escrito con su coste, porque la sección 4 cambia lo que es posible.

opción	qué es	veredicto
A — Invocación	SQX corre headless. La app manda órdenes y muestra progreso. SQX se abre solo para AlgoWizard y casos raros.	Es la que pide el equipo y es la correcta.
B — Empotrado real	Como la GUI de SQX es web (sección 4), se puede cargar en un <code>webview</code> dentro de la propia aplicación.	Ahora es viable , no como creía al principio. Útil como pestaña "SQX en crudo" sin salir de la ventana. Depende de abrir el acceso remoto.
C — Streaming	SQX en contenedor, servido por VNC en una pestaña.	Funciona y es feo. Solo tiene sentido para acceso remoto.

Recomendación: **A como producto**, **B como escotilla de escape** (una pestaña "Abrir SQX" que no obligue a cambiar de ventana), **C descartada** salvo que aparezca un requisito de acceso remoto.

12. Plan por fases

Con 1–2 ingenieros a tiempo completo, y sabiendo que el dominio ya está escrito. Las fases están ordenadas por *dolor eliminado por semana invertida*, no por dificultad.

fase	qué entrega	esfuerzo	qué dolor mata
0 • Demonio y cola	Un proceso. Los paneles actuales dejan de arrancar procesos y llaman al demonio. Cola unificada con progreso.	3–5 sem	P2 y la mitad de P3. Se acaba el "abre tres terminales"
1 • Carcasa y unificación visual	Ventana única, un sistema de diseño, los tres paneles reescritos como vistas con selección compartida	4–6 sem	P1. Es donde se gana el aspecto de producto
2 • Supervisor de SQX	Pool de workers headless, snapshots (R3), <code>on_done</code> encadenando export y análisis	6–8 sem	P3 entero. El botón "Walk Forward" con su barra
3 • Agente integrado	Sesión del agente dentro de la app, skills como herramientas, streaming y permisos	3–4 sem	El cambio de contexto que queda

≈ 4–6 meses hasta una v1 que alguien externo pueda usar sin manual.

La fase 2 es la delicada, por R3: es donde se puede destruir trabajo si se hace mal. La fase 0, en cambio, es de bajo riesgo y alto retorno, y es la que yo empezaría primero aunque el resto se aplace — porque deja el código mejor incluso si el producto nunca se construye.

⚠ Estas cifras son estimaciones de ingeniería, no medidas. A diferencia del resto del documento, no tienen evidencia detrás.

13. Riesgos

#	riesgo	gravedad	mitigación
1	Licencia de SQX ante orquestación headless y multi-instancia	puede matar el proyecto	Ya corréis maestro + worker, así que de facto funciona. Leer el EULA cuesta una tarde. Hacerlo antes de invertir nada.
2	SQX cambia de versión. El acoplamiento es por formatos de fichero, por <code>sqcli</code> y por el JS de su GUI	alta	Tests de contrato contra la instalación real. Sin ellos, cada actualización es una crisis
3	R3: destrucción de databanks por orquestación ingenua	alta	Snapshots obligatorios antes de cualquier reinicio. Es requisito de la fase 2, no un extra
4	Windows (R4)	media	Un día de trabajo para portar <code>sqx-worker.sh</code> a Python. Decidirlo pronto
5	Deuda declarada. <code>OPEN.md</code> tiene 22 hilos, varios de rigor estadístico (12, 13, 16, 19)	media, pero insidiosa	Una interfaz bonita sobre conclusiones no reproducibles es peor que una terminal fea. Cerrar 12 y 16 de la mano de la fase 0
6	El acceso remoto no expone lo esperado	baja	Existe la ruta B (log), que ya funciona

14. La prueba de concepto que zanja la pregunta

Antes de comprometer meses, hay una prueba de **2–3 días** que responde a todo lo que queda abierto:

Un botón, una barra, un Walk Forward real.

Alcance mínimo:

- 1. Demonio FastAPI mínimo, una sola ruta: `POST /job` y `GET /job/{id}/progress` (SSE).
- 2. `startOnlyTask` contra el worker en 5060, con un proyecto de prueba del worker.
- 3. Progreso leído por `tail` del log (ruta B, sin tocar configuración).
- 4. Una página con un botón y una barra.
- 5. En paralelo, abrir el acceso remoto y comprobar qué expone `/main/getWebSocketPort`.

Qué demuestra: que P3 es soluble, que la barra es real, y cuál de las dos rutas de progreso se queda. Qué no toca: el maestro, ningún proyecto del propietario, ningún databank.

15. Preguntas abiertas para el equipo

Ninguna bloquea la prueba de concepto, pero las cuatro cambian la arquitectura si se responden tarde:

1. **¿Quién lo usa?** ¿Solo el propietario, o también gente que no sabe qué es un databank? Decide cuánto hay que esconder y cuánta ayuda contextual hace falta.
 2. **¿Una máquina o varias?** ¿El demonio vive en el portátil de cada uno, o hay un servidor con SQX y clientes finos? **Esto decide la arquitectura entera y no se puede posponer.**
 3. **¿SQX tiene que seguir visible?** "El propietario quiere seguir abriendo su GUI cuando le apetezca" da un diseño distinto —y más fácil— que "que no se vea nunca".
 4. **¿Producto interno o vendible?** Interno permite atajos: rutas fijas, solo Linux, un solo idioma. Vendible, no.
-

Apéndice A — Evidencia reproducible

Todo lo empírico de este documento sale de estos comandos, ejecutados el 2026-09-20 con la GUI del maestro levantada y el worker apagado. Todos son de solo lectura.

```
# Puertos que SQX tiene abiertos (maestro con GUI: 5050 y 8080; el worker no estaba)
ss -ltnp | grep -E "5050|5060|8080"

# La GUI es Electron + Jetty
grep -oP 'INFO\s+\K[a-zA-Z0-9.]+' ~/Desktop/SQX/user/log/StrategyQuant/log_2026_09_19.log \
  | sort | uniq -c | sort -rn | head -15
ls ~/Desktop/SQX/internal/electron ~/Desktop/SQX/internal/web

# El canal de eventos que alimenta las barras de progreso de SQX
grep -rhoE ".{90}ws://.{90}" --include=*.js ~/Desktop/SQX/internal/web | head

# El endpoint, hoy cerrado
curl -s http://localhost:8080/main/getWebSocketPort
# → {"disabled":true,"error":"Remote access disabled"}

# El vocabulario de progreso del log
grep -oP 'ProgressEngine - \K.*' ~/Desktop/SQX/user/log/StrategyQuant/log_2026_09_19.log \
  | grep -viE "loaded -|saved -" | sed -E 's/[0-9]+/N/g' | sort -u

# El porcentaje, en el nombre del hilo
grep -oE '\[Blocking computeThread[^\]]*%[^\]]*\]' \
  ~/Desktop/SQX/user/log/StrategyQuant/log_2026_09_20.log | head

# La referencia completa de verbos de sqcli, sin arrancar SQX
cat ~/Desktop/SQX/internal/web/SQUANT/help.txt

# El inventario de AlgoProject
find core sqx tasks strategies perf tools -name "*.py" -not -path "**/sqx-lab/*" | wc -l
du -sh ~/Desktop/AlgoData/*
```

Apéndice B — Lo que NO se ha probado

Por honestidad, y para que nadie construya sobre arena:

- **¿ Qué expone SQX al abrir el acceso remoto.** Es la única incógnita relevante. Se resuelve en una tarde y decide si la barra de progreso es la de la ruta A o la de la ruta B.
- **¿ Si el WebSocket `/websocket/updates` emite progreso de tareas lanzadas por CLI,** o solo de las lanzadas desde su propia GUI. Plausible que sea lo primero —el evento nace en el motor, no en la interfaz— pero no verificado.
- **¿ Si un `webview` empotrado (opción B) sobrevive a las actualizaciones de SQX.** No probado.
- **¿ La forma exacta de la respuesta de `-project action=status`** en una tarea en marcha. No se arrancó el worker para no consumir un ciclo de la instalación.
- **⚠ Las estimaciones de esfuerzo de la sección 12** son juicio profesional, no medición.

Documento generado el 2026-09-20 a partir de una sesión de consultoría sobre el árbol en `master`. Los hallazgos de las secciones 4, 5 y 6 se incorporaron el mismo día a `knowhow/03-driving-sqx.md`, que es su sitio permanente; este documento es el relato completo, con el razonamiento y el plan. Si los dos discrepan, manda el knowhow.